Software Test Plan

for

Credit Card Fraud Detection and Model Performance Evaluation System

Version 1.1 (STP-CCFD-1.1)

Prepared by: Team Error 404

Date: 12-Oct-2026

Traced to: SRS-CCFD-1.2 · SADS-CCFD-1.1

Table of Contents

- Revision History
- 1. Introduction
- 2. Test Items
- 3. Features to Be Tested
  - 3.1 Features Not to Be Tested
- 4. Approach
  - 4.1 Effort Estimation Method
- 5. Test Cases
  - 5.1 Security Validation Test Cases
  - 5.2 Functional Test Cases
  - 5.3 Nonfunctional Test Cases
- 6. Effort Estimation
- 7. Team Distribution and Schedule
- 8. Item Pass/Fail Criteria
- 9. Suspension Criteria and Resumption Requirements
- 10. Test Deliverables
- 11. Environmental Needs
- 12. Responsibilities
- 13. Risks and Contingencies
- Appendix A: Requirements Traceability Matrix
- Appendix B: Glossary
- Project Team

Revision History

Name	Date	Reason for Changes	Version
Team Error 404	11-Oct-2026	Initial test plan for Release 1.0, traced to SRS v1.1.	1.0
Team Error 404	12-Oct-2026	Aligned with SRS v1.2 and SADS v1.1: corrected traceability of TC-SEC-04, TC-RES-07, TC-SAFE-04 and TC-API-05; added TC-SEC-10 (results retrieval requires the API key), TC-PERF-05 (NFR-05, NFR-Q03) and TC-PERF-06 (NFR-07); updated TC-PERF-01, TC-SEC-08, TC-SEC-09, TC-JOB-04, TC-JOB-05, TC-MOD-06, TC-MET-03 and TC-RES-04 for the clarified requirements; replaced the "implicit coverage" note with explicit test cases; recalculated all effort figures; added the test schedule (Section 7.3), features not to be tested (Section 3.1) and a requirement-centred traceability matrix; corrected the document identifier to STP-CCFD-1.1.	1.1

1. Introduction

This plan defines the testing that will verify that Release 1.0 of the Credit Card Fraud Detection and Model Performance Evaluation System satisfies its functional requirements (SRS Sections 3–4), its nonfunctional requirements (SRS Section 5: performance, safety, security and quality attributes) and its

business rules. The plan also estimates the effort needed for the 57 test cases and divides that work among the four team members over a three-day schedule.

Objectives:

1. Verify every High-priority SRS requirement through at least one test case.
2. Validate the security objectives and security requirements explicitly in Section 5.1.
3. Provide traceability from each test case to SRS requirement IDs, and from each SRS requirement to its test cases (Appendix A).
4. Estimate effort per test case and in total (Sections 4.1 and 6).
5. Distribute the test cases across the team, respecting dependencies, and schedule the work (Section 7).

In scope: the Web Client, the Flask Web Server and REST API, the three model microservices (ML, DL, QML), dataset upload and validation, asynchronous jobs, metrics, results presentation and CSV download.

No implementation has been built or executed at the time of writing. Expected results are specified from SRS v1.2 and the interfaces in SADS v1.1, and effort figures are estimates (see Sections 4.1 and 6 for the method and assumptions).

2. Test Items

The system under test (each build identified by its version-control commit hash at test time):

#	Test Item	Version / Note
TI-01	Web Client (HTML, CSS, JavaScript single-page UI)	Release 1.0
TI-02	Flask Web Server with REST API (`/api/datasets`, `/api/analyses`, `/api/analyses/{job_id}`)	Release 1.0, Python 3.10+, Flask 3.x
TI-03	ML microservice (POST /predict, GET /health), port 5001	Release 1.0
TI-04	DL microservice (POST /predict, GET /health), port 5002	Release 1.0
TI-05	QML microservice, simulator-based (POST /predict, GET /health), port 5003	Release 1.0
TI-06	Server-local dataset store (file system: datasets/, jobs/, results/)	Release 1.0

3. Features to Be Tested

SRS Feature	Requirement IDs	Test Cases
Security (objectives and requirements)	SO-01 – SO-03, SR-01 – SR-06	TC-SEC-01 – TC-SEC-10
Dataset upload and validation	REQ-FD-01 – REQ-FD-05, BR-01	TC-UPL-01 – TC-UPL-08, TC-SEC-05, TC-SEC-06, TC-RES-07
Model selection	REQ-FD-06 – REQ-FD-10, BR-02, BR-03	TC-MOD-01 – TC-MOD-06
Fraud detection execution (asynchronous jobs)	REQ-FD-11 – REQ-FD-14, BR-04	TC-JOB-01 – TC-JOB-06, TC-RES-03
Model performance evaluation	REQ-FD-15 – REQ-FD-17	TC-MET-01 – TC-MET-05

SRS Feature	Requirement IDs	Test Cases
Results presentation and user interface	REQ-FD-18 – REQ-FD-22, REQ-UI-01 – REQ-UI-06	TC-RES-01 – TC-RES-07, TC-MET-03, TC-JOB-02, TC-JOB-04
REST API operations	REQ-API-01 – REQ-API-05	TC-API-01 – TC-API-05
Performance and capacity	NFR-01 – NFR-07	TC-PERF-01 – TC-PERF-06, TC-UPL-01, TC-JOB-05
Safety	NFR-S01 – NFR-S03, BR-05	TC-SAFE-01 – TC-SAFE-03
Quality attributes	NFR-Q01 – NFR-Q05	TC-SAFE-04, TC-PERF-05, TC-MOD-06; NFR-Q02 and NFR-Q04 also by inspection (Appendix A)

3.1 Features Not to Be Tested

- Model retraining pipelines (out of scope per SRS Section 1.1).
- Deployment beyond a single host (SRS TBD-01) and TLS or public-internet hardening (SRS TBD-02).
- Physical quantum hardware; the QML service is tested only on its circuit simulator.
- The predictive quality of the trained models. Tests verify that metrics are computed and reported correctly, not that a model reaches a particular accuracy.

4. Approach

Test levels.

1. **Unit / component:** each microservice in isolation against its HTTP contract, using datasets placed in the dataset store by a test fixture.
2. **Integration:** Web Server ↔ microservices (routing, job lifecycle, failure propagation) and Web Server ↔ dataset store (storage, permissions, retention).
3. **System:** end-to-end through the Web Client and through the REST API directly.

Techniques. Black-box functional testing; boundary-value analysis (file size, row count, label values, pagination at 50/51 rows); negative testing (malformed, oversized, mislabeled inputs); security testing (authentication bypass, schema violations, information disclosure, file permissions); performance measurement (timers, 10-client polling and session harnesses); a usability walkthrough.

Test data. Synthetic labeled CSVs only, with no real cardholder data (NFR-S03). All valid files contain the `Class` column and the feature columns listed in the documented dataset schema.

File	Content
valid_1000.csv	1,000 valid rows, Class 0/1
perf_10k.csv	10,000 valid rows (NFR-01 upper bound)
perf_100k.csv	100,000 valid rows (NFR-03)
valid_51.csv	51 valid rows (pagination boundary)
tiny_4row.csv	4 rows with hand-computable metrics
no_class_column.csv	valid file without the Class column
bad_labels.csv	Class values 2, −1 and empty

File	Content
header_only.csv / empty.csv	header row only / 0 bytes
semicolon.csv	otherwise valid, semicolon-delimited
non_numeric.csv	text value in a required feature column
extra_cols.csv	valid file plus two extra columns, one a synthetic `card_ref` text column
large_noclass.csv	≈400 MB file without the Class column (NFR-07)
evil.csv	binary executable renamed with a .csv extension
reference_metrics.json	independently computed metrics for valid_1000.csv

Entry criteria. All test items deploy on the reference machine; GET /health returns 200 on every microservice; the test API key is configured; test datasets are prepared.

Smoke test. GET /health → 200 on all three services, TC-UPL-01 passes, and TC-JOB-02 reaches completed. To verify NFR-Q04 (portability), the smoke test is run once on Windows 10/11 and once on Ubuntu 20.04+.

Tools. pytest and the `requests` library (API-level cases), curl (manual API checks, `-w %{time_total}` timing), browser developer tools (UI and rendering time), a 10-client polling script (TC-PERF-02) and a 10-session script (TC-PERF-05).

Regression. After any defect fix: re-run all High-priority cases plus every previously failed case.

4.1 Effort Estimation Method

Each test case carries an *Est.* value: the estimated hands-on time for one tester to set up, execute, verify and log that case, in minutes. Estimates are calibrated by case type from the team's earlier lab experience:

Case Type	Minutes	Rationale
Simple API probe (single request, assert status/body)	10–15	curl + assert, no setup
API flow (multi-step: upload → run → poll)	20–30	Setup plus waiting on async jobs
Negative / boundary case	15–20	Fixture preparation dominates
Security inspection (logs, permissions, retention)	25–30	Manual inspection steps
UI flow	20–25	Browser interaction and observation
Performance measurement	25–45	Harness setup, multiple runs, median of 3
Failure injection (stop service, timeouts)	30–40	Environment manipulation and waiting

Overheads added on top of pure execution: environment setup (one-time), a defect-fix buffer, regression re-runs, and reporting. Section 6 totals everything; Section 7 distributes it.

5. Test Cases

Each test case lists its priority, type and estimated effort, the SRS requirements it verifies (Traceability), preconditions, steps, expected result and pass criterion. Unless stated otherwise, the precondition is: all services healthy and every request carries `Authorization: Bearer <test key>`.

5.1 Security Validation Test Cases

TC-SEC-01 — Dataset upload without API key → 401, nothing stored	
Priority / Type / Est.	High · Security · 20 min
Objective	The upload endpoint rejects unauthenticated callers and stores nothing.
Traceability	SR-03, REQ-API-04 (objective SO-02)
Preconditions	Server healthy; Authorization header deliberately omitted.
Steps	1. POST /api/datasets with valid_1000.csv and no Authorization header. 2. List the datasets/ directory of the dataset store.
Expected	HTTP 401 with a {code, message} JSON body; the store is unchanged.
Pass	401 received and no new file in the store.

TC-SEC-02 — Invalid API key → 401	
Priority / Type / Est.	High · Security · 15 min
Objective	A wrong key is treated the same as a missing key.
Traceability	SR-03
Steps	1. POST /api/datasets with valid_1000.csv and header `Authorization: Bearer invalid-key-123`.
Expected	HTTP 401; nothing stored.
Pass	401 received; store unchanged.

TC-SEC-03 — Analysis start without API key → 401, no job created	
Priority / Type / Est.	High · Security · 20 min
Objective	Job creation is protected; no phantom jobs appear.
Traceability	SR-03, REQ-API-02 (objective SO-02)
Preconditions	A valid dataset_id from TC-UPL-01.
Steps	1. POST /api/analyses with `{"dataset_id":"<id>","model_type":"ml"}` and no Authorization header. 2. List the jobs/ directory of the dataset store and the microservice access logs.
Expected	HTTP 401; no job record created; no microservice request.
Pass	401 received; no new job record; no new microservice request.

TC-SEC-04 — Malformed JSON rejected before model processing, never logged	
Priority / Type / Est.	High · Security · 25 min
Objective	Strict schema validation stops malformed input at the server; payload contents never reach logs or microservices.
Traceability	SR-02, REQ-API-04 (objective SO-03)
Preconditions	A valid dataset_id.
Steps	1. POST /api/analyses with the truncated body `{"dataset_id":`. 2. POST /api/analyses with `{"dataset_id":"<id>"}` (model_type missing). 3. POST /api/analyses with `{"dataset_id":"<id>","model_type":"ml","debug":true}` (unknown field). 4. Inspect the application log and the microservice access logs.
Expected	HTTP 400 with a reason-naming body for all three requests; no job created; no payload fragment in any log; no microservice activity.
Pass	400 on all three; logs contain no payload fragments; microservices show no new requests.

TC-SEC-05 — Executable disguised as CSV rejected	
Priority / Type / Est.	High · Security · 20 min
Objective	File-type validation blocks non-CSV content regardless of the extension.
Traceability	SR-01, REQ-FD-04
Steps	1. POST /api/datasets with evil.csv (a binary executable renamed to .csv).
Expected	HTTP 400 naming "unsupported file type"; nothing stored; no microservice contacted.
Pass	400 with the specific reason; store unchanged.

TC-SEC-06 — Oversize upload rejected with specific reason	
Priority / Type / Est.	Medium · Security · 25 min
Objective	The 500 MB limit is enforced with a clear message.
Traceability	SR-01, REQ-FD-02, REQ-FD-04
Steps	1. Upload a file larger than 500 MB. If this is impractical in lab time, temporarily lower the configured limit (for example to 5 MB), upload a larger file and record the substitution.
Expected	HTTP 400 with "file exceeds the 500 MB limit" (or the substituted limit); nothing stored.
Pass	400 with the size reason; store unchanged.

TC-SEC-07 — Error responses disclose no internals	
Priority / Type / Est.	High · Security · 25 min
Objective	Client-facing errors and job failure reasons never leak stack traces, paths or configuration.
Traceability	SR-06 (objectives SO-01, SO-02)

Steps	1. GET /api/analyses/{random-uuid} → expect 404. 2. POST /api/analyses with malformed JSON → expect 400. 3. Make the dataset store directory read-only, upload valid_1000.csv → expect 500, then restore permissions. 4. Read the failure reason of a failed job from TC-JOB-05.
Expected	Every body contains only {code, message} (or the job's reason object); no `Traceback`, file-system path, `.py` reference or configuration value.
Pass	None of the forbidden strings appear in any response.

TC-SEC-08 — No transaction data in logs, UI or export	
Priority / Type / Est.	High · Security · 30 min
Objective	Transaction field values are never logged, displayed or exported (data display policy).
Traceability	SR-04 (objective SO-01)
Preconditions	A completed ML job on extra_cols.csv (it contains a synthetic `card_ref` column).
Steps	1. Search the application and microservice logs for distinctive values from extra_cols.csv (feature values and card_ref strings). 2. Inspect the results table and the downloaded CSV. 3. Upload non_numeric.csv and read the error message.
Expected	No field value appears in any log; the table and CSV show only row index, predicted class and fraud probability (no card_ref or feature column); the error message names the column and row but not the cell value.
Pass	All three checks hold.

TC-SEC-09 — Dataset store permissions and 24-hour retention	
Priority / Type / Est.	Medium · Security · 30 min
Objective	Stored data is isolated on disk and deleted when its retention period ends.
Traceability	SR-05 (objective SO-01)
Steps	1. Upload valid_1000.csv and complete one job on it. 2. Check the permissions of the dataset store directory and its files. 3. With the retention period temporarily configured to 5 minutes (substitution recorded), wait until it has elapsed. 4. List datasets/, jobs/ and results/; GET the job.
Expected	Owner-only permissions for the service account; after the retention period the dataset file, job record and result file are deleted and GET returns 404.
Pass	Permissions are owner-only and all three artefacts are removed after the period ends.

TC-SEC-10 — Results retrieval without API key → 401	
Priority / Type / Est.	High · Security · 15 min
Objective	Results of a job cannot be read without a valid key, even when the job_id is known.
Traceability	SR-03, REQ-API-03 (objectives SO-01, SO-02)
Preconditions	A completed job from TC-JOB-02.

Steps	1. GET /api/analyses/{job_id} with no Authorization header. 2. Repeat with `Authorization: Bearer invalid-key-123`.
Expected	HTTP 401 for both; no predictions or metrics in the body.
Pass	401 on both requests and no result data returned.

5.2 Functional Test Cases

Dataset upload and validation

TC-UPL-01 — Valid dataset accepted and registered	
Priority / Type / Est.	High · Functional · 15 min
Objective	A well-formed labeled CSV is accepted with a unique dataset identifier.
Traceability	REQ-FD-02, REQ-FD-03, REQ-FD-05, REQ-API-01, NFR-01
Steps	1. POST /api/datasets with valid_1000.csv; record status, body and elapsed time. 2. Upload the same file again.
Expected	HTTP 201 with a non-empty dataset_id and rows = 1000, returned within 2 s; the second upload receives a different dataset_id.
Pass	201 with dataset_id; elapsed ≤ 2 s; the two identifiers differ.

TC-UPL-02 — Missing Class column → 400 naming the reason	
Priority / Type / Est.	High · Functional · 15 min
Objective	Schema validation catches the missing label column with a specific message.
Traceability	REQ-FD-03, REQ-FD-04, BR-01
Steps	1. Upload no_class_column.csv.
Expected	HTTP 400; message "missing required column: Class".
Pass	400 with the column named; nothing stored.

TC-UPL-03 — Non-binary Class values → 400	
Priority / Type / Est.	Medium · Functional · 15 min
Objective	The label domain (0/1 only) is enforced.
Traceability	REQ-FD-03, REQ-FD-04
Steps	1. Upload bad_labels.csv (Class values 2, −1 and empty).
Expected	HTTP 400 naming the label-domain violation.
Pass	400 with a specific reason.

TC-UPL-04 — Header-only file → 400

Priority / Type / Est.	Medium · Functional · 15 min
Objective	A file with no data rows is rejected.
Traceability	REQ-FD-03, REQ-FD-04
Steps	1. Upload header_only.csv.
Expected	HTTP 400 stating that the file has no data rows.
Pass	400 with a specific reason.

TC-UPL-05 — Empty file → 400	
Priority / Type / Est.	Low · Functional · 10 min
Objective	A zero-byte upload is rejected gracefully.
Traceability	REQ-FD-04
Steps	1. Upload empty.csv (0 bytes). 2. Repeat TC-UPL-01 step 1.
Expected	HTTP 400; the server keeps working.
Pass	400; the subsequent valid upload still returns 201.

TC-UPL-06 — Wrong delimiter → 400	
Priority / Type / Est.	Medium · Functional · 15 min
Objective	Only comma-delimited CSV is accepted.
Traceability	REQ-FD-02, REQ-FD-04
Steps	1. Upload semicolon.csv.
Expected	HTTP 400 naming the format problem as the reason.
Pass	400 with a specific reason.

TC-UPL-07 — Non-numeric feature value → 400	
Priority / Type / Est.	Medium · Functional · 15 min
Objective	Required feature columns must be numeric.
Traceability	REQ-FD-03, REQ-FD-04
Steps	1. Upload non_numeric.csv.
Expected	HTTP 400 naming the offending column.
Pass	400 with the column named.

TC-UPL-08 — Extra columns tolerated and ignored	
Priority / Type / Est.	Low · Functional · 15 min

Objective	Files with columns beyond the documented schema are accepted and the extra columns are ignored.
Traceability	REQ-FD-03
Steps	1. Upload extra_cols.csv. 2. Run an ML job on it and poll to completion.
Expected	HTTP 201; the run completes; predictions cover all rows.
Pass	201 and a completed run with the correct row count.

Model selection and routing

TC-MOD-01 — Run blocked with no model selected	
Priority / Type / Est.	High · Functional · 15 min
Objective	The UI enforces model selection before execution.
Traceability	REQ-FD-06, BR-02
Preconditions	Web Client open; API key entered; a dataset uploaded.
Steps	1. Leave the model selector empty and activate the run control.
Expected	The run does not start; a message asks the user to select a model.
Pass	No POST /api/analyses is sent (browser network panel); the guidance message is shown.

TC-MOD-02 — ML selection routes to the ML microservice	
Priority / Type / Est.	High · Functional · 20 min
Objective	Selecting ML executes the ML service and labels results accordingly.
Traceability	REQ-FD-06, REQ-FD-07, REQ-FD-10, REQ-FD-11, REQ-FD-17, BR-03
Steps	1. POST /api/analyses with `{"dataset_id":"<id>","model_type":"ml"}`. 2. Poll to completion; check the ML service access log and the model_name.
Expected	202 with job_id; the ML service received the request; the completed job's model_name is the classifier configured for the ML service.
Pass	202; ML service called; model_name matches the configuration.

TC-MOD-03 — DL selection routes to the DL microservice	
Priority / Type / Est.	High · Functional · 20 min
Objective	As TC-MOD-02 for the DL option.
Traceability	REQ-FD-06, REQ-FD-08, REQ-FD-10, REQ-FD-11, REQ-FD-17, BR-03
Steps	1. Run with model_type "dl"; poll to completion; check the DL access log and model_name.
Expected	202; completed job; model_name identifies the DL (MLP) model.
Pass	202; DL service called; model_name correct.

TC-MOD-04 — QML selection routes to the QML microservice	
Priority / Type / Est.	High · Functional · 25 min
Objective	As TC-MOD-02 for the simulator-backed QML option.
Traceability	REQ-FD-06, REQ-FD-09, REQ-FD-10, REQ-FD-11, REQ-FD-17, BR-03
Steps	1. Run with model_type "qml" on valid_1000.csv; poll to completion.
Expected	202; completed job (the simulator may be slower, which the asynchronous design absorbs); model_name identifies the QML model.
Pass	202; completed with the QML model name.

TC-MOD-05 — Unknown model_type → 400	
Priority / Type / Est.	Medium · Functional · 15 min
Objective	The model-type enumeration is enforced at the API boundary.
Traceability	REQ-FD-06, REQ-API-02, REQ-API-04, SR-02
Steps	1. POST /api/analyses with model_type "quantum-xyz".
Expected	HTTP 400 naming the invalid model type; no job created.
Pass	400; no job record exists.

TC-MOD-06 — Microservice contract tested in isolation	
Priority / Type / Est.	Medium · Functional · 20 min
Objective	Each microservice honours the uniform contract on its own, without the Web Server.
Traceability	REQ-FD-10, NFR-Q05
Preconditions	valid_1000.csv placed in the dataset store under a known dataset_id by the test fixture.
Steps	1. GET /health on ports 5001, 5002 and 5003. 2. POST /predict with the known dataset_id to each service. 3. POST /predict with an unknown dataset_id.
Expected	200 `{"status":"ok"}` from each health check; 200 with model_name, predictions and metrics in the contract shape; 404 with {code, message} for the unknown identifier.
Pass	All three services satisfy all three checks.

Asynchronous job execution

TC-JOB-01 — Valid run accepted within 2 s with job_id	
Priority / Type / Est.	High · Functional · 15 min
Objective	Analysis runs are asynchronous: the server accepts and returns at once.
Traceability	REQ-FD-12, REQ-API-02
Steps	1. POST /api/analyses with `{"dataset_id":"<id>","model_type":"ml"}`; measure the time to response.

Expected	HTTP 202 within 2 s with a non-empty job_id and status "queued".
Pass	202 within 2 s; status "queued".

TC-JOB-02 — Poll observes queued → running → completed in order	
Priority / Type / Est.	High · Functional · 30 min
Objective	The full job lifecycle is observable and ordered; the client needs no reload.
Traceability	REQ-FD-12, REQ-UI-03
Steps	1. Start an ML run from the Web Client. 2. In the browser network panel, record each GET /api/analyses/{job_id} response and its time; watch the status badge.
Expected	Polls every 2 s; states progress queued → running → completed; the badge updates without a reload; polling stops at the terminal state.
Pass	All three states seen in order, polling interval 2 s (± 0.5 s), no manual refresh, no polls after completion.

TC-JOB-03 — Unknown job_id → 404	
Priority / Type / Est.	Medium · Functional · 10 min
Objective	Polling a nonexistent job returns a clean 404.
Traceability	REQ-API-03, REQ-API-04
Steps	1. GET /api/analyses/{random-uuid}.
Expected	HTTP 404 with a {code, message} body.
Pass	404 with the standard error shape.

TC-JOB-04 — Two concurrent jobs stay independent	
Priority / Type / Est.	Medium · Functional · 30 min
Objective	Concurrent jobs do not interfere and each result set stays with its own dataset and model.
Traceability	REQ-FD-11, REQ-FD-12, REQ-FD-22
Steps	1. Upload valid_1000.csv and valid_51.csv. 2. Start an ML run on the first and a DL run on the second back-to-back. 3. Poll both to completion; GET both jobs again 30 minutes later.
Expected	Two distinct job_ids; both complete; each job reports its own dataset_id, model_type and model_name, with 1,000 and 51 predictions respectively, and still does so later in the retention period.
Pass	Both complete with correctly associated results; no cross-contamination.

TC-JOB-05 — Microservice down → job failed; other services unaffected	
Priority / Type / Est.	High · Functional (negative) · 35 min

Objective	An unavailable service yields a failed job with a reason — never fabricated results — and the other services keep working.
Traceability	REQ-FD-14, REQ-FD-20, BR-04, NFR-06
Preconditions	QML microservice stopped.
Steps	1. Start a run with model_type "qml" from the Web Client. 2. Poll until a terminal state. 3. Start an ML run on the same dataset.
Expected	The QML job becomes failed with reason MODEL_SERVICE_ERROR; the Web Client shows the reason and no predictions or metrics; the ML job completes normally.
Pass	failed with reason and no results shown; ML run completes.

TC-JOB-06 — Job timeout → failed with reason	
Priority / Type / Est.	Medium · Functional (negative) · 40 min
Objective	The 600 s limit fires; hung jobs cannot linger.
Traceability	REQ-FD-14
Preconditions	A stub microservice that never responds, registered as the ML service.
Steps	1. Start a run against the stub. 2. Poll past the configured timeout (a lower value, for example 30 s, may be configured; record the substitution).
Expected	The job transitions to failed with reason JOB_TIMEOUT.
Pass	failed with JOB_TIMEOUT recorded.

Model performance evaluation

TC-MET-01 — Metrics correct to four decimals against an independent reference	
Priority / Type / Est.	High · Functional · 30 min
Objective	The five metrics match an independently computed reference.
Traceability	REQ-FD-15
Preconditions	Completed ML job on valid_1000.csv; reference_metrics.json computed by a separate script from the same predictions.
Steps	1. Read the metrics from GET /api/analyses/{job_id}. 2. Compare each value with the reference.
Expected	accuracy, precision, recall, f1 and roc_auc present, each with four decimal places, matching the reference.
Pass	All five match to 4 d.p.

TC-MET-02 — Confusion-matrix counts are consistent	
Priority / Type / Est.	High · Functional · 15 min
Objective	The confusion matrix is complete and arithmetically consistent.

Traceability	REQ-FD-16
Steps	1. Read confusion_matrix from a completed job.
Expected	Integer tp, tn, fp and fn whose sum equals the dataset row count.
Pass	tp + tn + fp + fn equals the input row count.

TC-MET-03 — Metrics and confusion matrix displayed and labeled with the executed model	
Priority / Type / Est.	High · Functional/UI · 20 min
Objective	The results screen shows the metrics and confusion matrix, always identified by their source model.
Traceability	REQ-FD-17, REQ-FD-19, BR-03
Steps	1. Complete one run per model type from the Web Client. 2. Inspect each results screen and metrics payload.
Expected	Each screen shows the five metrics and the four confusion-matrix counts, matching the API payload, labeled with the model_name of that job.
Pass	All three runs display complete, correctly labeled metrics.

TC-MET-04 — Same dataset across ML, DL and QML yields comparable metric sets	
Priority / Type / Est.	Medium · Functional · 30 min
Objective	Comparing approaches on identical data works end to end.
Traceability	REQ-FD-15, REQ-FD-16
Steps	1. Run all three model types on valid_1000.csv. 2. Collect the three metric sets.
Expected	Three complete metric sets with identical structure, computed over the same row count.
Pass	All three complete; structures identical.

TC-MET-05 — Hand-computed tiny dataset matches exactly	
Priority / Type / Est.	Medium · Functional · 25 min
Objective	End-to-end metric correctness on a dataset small enough to verify by hand.
Traceability	REQ-FD-15, REQ-FD-16
Steps	1. Upload tiny_4row.csv, run ML, read the predictions, metrics and confusion matrix. 2. Compute the metrics by hand from the returned predictions and the labels.
Expected	Values match the hand computation (to 4 d.p.).
Pass	Exact match on all five metrics and all four counts.

Results presentation and user interface

TC-RES-01 — Predictions table: columns and 50-row pagination

Priority / Type / Est.	High · Functional/UI · 20 min
Objective	The results table shows the required columns, paginated.
Traceability	REQ-FD-18, REQ-UI-05
Preconditions	Completed job on valid_1000.csv.
Steps	1. Open the results view; inspect the columns and the pager.
Expected	Columns row index, predicted class and fraud probability; 50 rows per page; 20 pages for 1,000 rows.
Pass	Columns correct; 50 rows per page; 20 pages.

TC-RES-02 — Pagination boundary: 51 rows → 2 pages	
Priority / Type / Est.	Medium · Functional/UI · 20 min
Objective	The page boundary drops or duplicates no rows.
Traceability	REQ-FD-18, REQ-UI-05
Steps	1. Run on valid_51.csv; open the results; count the rows on pages 1 and 2.
Expected	Page 1 has 50 rows and page 2 has 1 row; row indices are continuous.
Pass	50 + 1 rows with no gaps or duplicates.

TC-RES-03 — Prediction value domains	
Priority / Type / Est.	High · Functional · 15 min
Objective	Every row receives a valid class and probability.
Traceability	REQ-FD-13
Steps	1. Read the full predictions array of a completed job on valid_1000.csv.
Expected	One prediction per input row; class ∈ {0, 1}; probability ∈ [0, 1].
Pass	Count matches the input; all values in domain.

TC-RES-04 — Results appear automatically when the job completes	
Priority / Type / Est.	High · UI · 20 min
Objective	The Web Client moves from status to results on its own.
Traceability	REQ-UI-03, REQ-FD-20
Steps	1. Start a run in the browser; do not reload or click anything further.
Expected	The status badge moves through queued → running → completed, and the predictions table and metrics appear automatically on completion.
Pass	Results shown with no reload or extra user action.

TC-RES-05 — Error message adjacent to control within 1 s	
Priority / Type / Est.	High · UI · 20 min
Objective	Errors are shown where they happened, promptly.
Traceability	REQ-UI-02
Steps	1. In the browser, upload no_class_column.csv; in the performance panel, measure from receipt of the 400 response to the message appearing; note its position.
Expected	The message appears next to the upload control within 1 s of the server response.
Pass	Adjacent placement and ≤ 1 s.

TC-RES-06 — CSV download: columns and row count	
Priority / Type / Est.	Medium · Functional · 20 min
Objective	Completed predictions are downloadable in the specified shape.
Traceability	REQ-FD-21
Preconditions	Completed job on valid_1000.csv.
Steps	1. Use the download control; open the CSV.
Expected	Columns row index, predicted class and fraud probability; one row per transaction.
Pass	Exactly the three columns; 1,000 data rows.

TC-RES-07 — Single-page layout, upload control, help link and 1280×720 viewport	
Priority / Type / Est.	High · UI · 25 min
Objective	All controls are on one page, the upload control takes one file, guidance is always reachable and the layout works at the minimum viewport.
Traceability	REQ-UI-01, REQ-UI-04, REQ-UI-06, REQ-FD-01
Steps	1. Open the Web Client; confirm the API-key field, upload control, model selector, run control and results region are on one page. 2. Try to select two files in the file chooser. 3. Check the help link in every state of the page and open it. 4. Resize the viewport to 1280×720 and walk the upload → run → results flow.
Expected	All controls on one page; the chooser accepts a single file; the help link is visible in every state and opens the manual; all controls reachable with consistent styling at 1280×720.
Pass	All four checks hold.

REST API contract

TC-API-01 — POST /api/datasets contract	
Priority / Type / Est.	High · Functional · 10 min
Objective	The upload endpoint's success contract is exact.
Traceability	REQ-API-01, REQ-API-05

Steps	1. POST /api/datasets with valid_1000.csv; inspect status, headers and body.
Expected	HTTP 201; Content-Type application/json; body `{"dataset_id": "<non-empty>", "rows": 1000}`.
Pass	201 with the exact body shape and JSON content type.

TC-API-02 — POST /api/analyses contract	
Priority / Type / Est.	High · Functional · 10 min
Objective	The analysis endpoint's acceptance contract is exact.
Traceability	REQ-API-02, REQ-API-05
Steps	1. POST /api/analyses with a valid body.
Expected	HTTP 202; body `{"job_id": "<non-empty>", "status": "queued"}`.
Pass	202 with the exact body shape.

TC-API-03 — GET /api/analyses/{job_id} contract	
Priority / Type / Est.	High · Functional · 15 min
Objective	Status polling returns the documented job shape in every state.
Traceability	REQ-API-03, REQ-API-05
Steps	1. GET a running job, a completed job and a failed job (from TC-JOB-05); inspect the bodies.
Expected	200 with job_id, dataset_id, model_type and status in every state; model_name, predictions and metrics only when completed; reason {code, message} only when failed.
Pass	Shape correct in all three states.

TC-API-04 — Uniform error body	
Priority / Type / Est.	Medium · Functional · 20 min
Objective	All errors share the uniform body shape.
Traceability	REQ-API-04
Steps	1. Trigger 400 (bad upload), 401 (no key) and 404 (unknown job). 2. Inspect each body.
Expected	Each body is `{"code": "<MACHINE_CODE>", "message": "<text>"}` with JSON content type, using the codes listed in SADS Section 3.3.
Pass	Uniform shape and documented codes on all three.

TC-API-05 — Wrong HTTP method → 405	
Priority / Type / Est.	Low · Functional · 10 min
Objective	Method enforcement on the API surface.

Traceability	REQ-API-04, REQ-API-05
Steps	1. GET /api/datasets. 2. DELETE /api/analyses/{job_id}.
Expected	HTTP 405 with the uniform error body for both.
Pass	405 and error body on both.

5.3 Nonfunctional Test Cases

Performance and capacity

TC-PERF-01 — Upload result within 2 s for up to 10,000 rows	
Priority / Type / Est.	High · Performance · 30 min
Objective	Typical uploads are acknowledged promptly.
Traceability	NFR-01
Preconditions	Single-host deployment on the reference machine; client on the same host so transfer time is negligible.
Steps	1. Upload valid_1000.csv and perf_10k.csv with `curl -w %{time_total}`; repeat each 3 times. 2. Repeat with a 10,000-row file that has a missing Class column.
Expected	Each 201 or 400 response arrives within 2 s.
Pass	Median ≤ 2 s for every file.

TC-PERF-02 — Status polling p95 ≤ 500 ms under 10 concurrent pollers	
Priority / Type / Est.	High · Performance · 45 min
Objective	The status endpoint stays responsive under concurrent polling.
Traceability	NFR-02
Preconditions	A running job; the 10-client polling script.
Steps	1. Launch 10 concurrent pollers for 60 s against GET /api/analyses/{job_id}. 2. Compute the response-time distribution.
Expected	p95 response time ≤ 500 ms; no errors.
Pass	p95 ≤ 500 ms and zero 5xx responses.

TC-PERF-03 — 100,000-row validation and storage within 60 s	
Priority / Type / Est.	Medium · Performance · 45 min
Objective	Large-dataset intake meets its bound on the reference machine.
Traceability	NFR-03
Steps	1. Upload perf_100k.csv; measure from upload completion (server log timestamp) to the 201 response; repeat 3 times on an idle machine.
Expected	Validation and storage complete within 60 s.

Pass	Median ≤ 60 s.

TC-PERF-04 — Results page renders within 3 s	
Priority / Type / Est.	Medium · Performance · 20 min
Objective	Result rendering is prompt after the completed response arrives.
Traceability	NFR-04
Preconditions	Completed job on valid_1000.csv; browser developer tools.
Steps	1. Measure from receipt of the completed job response to the rendered table and metrics; repeat 3 times.
Expected	Full render within 3 s.
Pass	Median ≤ 3 s.

TC-PERF-05 — 10 concurrent analysis sessions	
Priority / Type / Est.	Medium · Performance · 45 min
Objective	The system sustains 10 parallel sessions without breaking its other targets.
Traceability	NFR-05, NFR-Q03
Preconditions	The 10-session script: each session uploads valid_1000.csv, starts a run (model types spread across ML, DL and QML) and polls every 2 s to completion.
Steps	1. Run 10 sessions concurrently. 2. Record upload times, status-poll latencies, response codes and, for one session, the browser render time.
Expected	Upload times ≤ 2 s, poll p95 ≤ 500 ms, render ≤ 3 s; at least 99% of requests receive the correct status code; all jobs reach a terminal state.
Pass	All targets met under the 10-session load.

TC-PERF-06 — Fast rejection of a large structurally invalid file	
Priority / Type / Est.	Medium · Performance · 25 min
Objective	File-level errors are reported quickly regardless of file size.
Traceability	NFR-07
Steps	1. Upload large_noclass.csv (≈400 MB, no Class column); measure from upload completion (server log timestamp) to the 400 response; repeat 3 times.
Expected	HTTP 400 "missing required column: Class" within 5 s of upload completion.
Pass	Median ≤ 5 s with the correct reason.

Safety and usability

TC-SAFE-01 — Analytical-only notice on every results screen

Priority / Type / Est.	High · Safety · 15 min
Objective	Outputs are framed as analytical, never as authorization decisions.
Traceability	NFR-S02, BR-05
Steps	1. Complete a job; inspect every results state (table pages, metrics, download area) for the notice.
Expected	"Analytical result only — not a financial authorization decision." is visible on all results screens.
Pass	Notice present everywhere results appear.

TC-SAFE-02 — No transaction-initiating controls exist	
Priority / Type / Est.	High · Safety · 15 min
Objective	The system cannot initiate, authorize or block real financial transactions.
Traceability	NFR-S01, BR-05
Steps	1. Walk the entire UI and the API route list (SADS Section 3.2) looking for any control or endpoint that triggers a transaction.
Expected	No such control or endpoint exists.
Pass	None found.

TC-SAFE-03 — Sample datasets contain no real cardholder data	
Priority / Type / Est.	Medium · Safety · 20 min
Objective	Shipped demonstration data is synthetic or anonymized.
Traceability	NFR-S03
Steps	1. Inspect every sample dataset shipped with the system for card-number-like or real-looking cardholder values.
Expected	All values synthetic or anonymized and documented as such.
Pass	No real cardholder data found.

TC-SAFE-04 — First-time user completes the workflow within 5 minutes unaided	
Priority / Type / Est.	Medium · Usability · 30 min
Objective	A first-time user completes the full flow without help.
Traceability	NFR-Q01
Preconditions	A participant who has never operated the UI (for example a classmate outside the team); a timer; the API key written on a card.
Steps	1. Without assistance, the participant enters the API key, uploads valid_1000.csv, selects a model, runs the analysis and views the results.
Expected	Completion within 5 minutes with no assistance.

Pass	Completed within 5 minutes unaided.

6. Effort Estimation

6.1 Per-area totals

Pure execution effort (set up, execute, verify and log each case, from the Est. values):

Area	Cases	Execution effort
Security (TC-SEC-01 – 10)	10	20 + 15 + 20 + 25 + 20 + 25 + 25 + 30 + 30 + 15 = 225 min (3.75 h)
Upload & validation (TC-UPL-01 – 08)	8	15 + 15 + 15 + 15 + 10 + 15 + 15 + 15 = 115 min (1.92 h)
Model selection (TC-MOD-01 – 06)	6	15 + 20 + 20 + 25 + 15 + 20 = 115 min (1.92 h)
Job lifecycle (TC-JOB-01 – 06)	6	15 + 30 + 10 + 30 + 35 + 40 = 160 min (2.67 h)
Metrics (TC-MET-01 – 05)	5	30 + 15 + 20 + 30 + 25 = 120 min (2.00 h)
Results & UI (TC-RES-01 – 07)	7	20 + 20 + 15 + 20 + 20 + 20 + 25 = 140 min (2.33 h)
API contract (TC-API-01 – 05)	5	10 + 10 + 15 + 20 + 10 = 65 min (1.08 h)
Performance (TC-PERF-01 – 06)	6	30 + 45 + 45 + 20 + 45 + 25 = 210 min (3.50 h)
Safety & usability (TC-SAFE-01 – 04)	4	15 + 15 + 20 + 30 = 80 min (1.33 h)
Total execution	**57**	**1230 min = 20.50 h**

6.2 Overheads

Item	Effort	Note
Environment setup and smoke test	2.0 h	One-time: machine provisioning, dataset preparation, API-key fixture
Defect-fix buffer	5.0 h	About 25% of execution effort (20.50 h × 0.25 ≈ 5.12 h, rounded to 5.0 h so it divides evenly: 1.25 h per member)
Regression re-runs	2.0 h	All High-priority cases plus previously failed cases after fixes
Summary report and traceability review	2.0 h	Test summary, pass-rate computation, release recommendation
Total overhead	**11.0 h**	

6.3 Total

Execution 20.50 h + overhead 11.0 h = **31.50 person-hours** for the full test campaign.

Assumptions: the implementation is complete and deployable at the start; one reference machine is available; the documented substitutions for the 600 s timeout (TC-JOB-06), the 500 MB limit (TC-SEC-06) and the 24-hour retention period (TC-SEC-09) are accepted; QML simulator run times stay within

the asynchronous design's tolerance. If the build is not yet stable, one more fix-and-retest cycle (about 6 h) is added.

7. Team Distribution and Schedule

7.1 Assignment

Member	Role	Test cases owned	Case effort	Other duties	Total
Akshat Agarwal (PES1UG24AM024)	Test lead	TC-SEC-01 – TC-SEC-10 (security validation); defect triage	225 min = 3.75 h	Environment setup (2.0 h); summary report (2.0 h); defect-fix share (1.25 h)	9.00 h
Aman Priyansh (PES1UG24AM030)	Test executor	TC-UPL-01 – 08, TC-API-01 – 05, TC-PERF-01 – 06	390 min = 6.50 h	defect-fix share (1.25 h)	7.75 h
Aditya Kalloli (PES1UG24AM015)	Test executor	TC-MOD-01 – 06, TC-JOB-01 – 06	275 min = 4.58 h	Regression re-runs (2.0 h); defect-fix share (1.25 h)	7.83 h
Raghav Balaji (PES1UG23AM224)	Test executor	TC-RES-01 – 07, TC-MET-01 – 05, TC-SAFE-01 – 04	340 min = 5.67 h	defect-fix share (1.25 h)	6.92 h
Total	| **57 cases**	**20.50 h**	**11.00 h**	**31.50 h**	

Each member's total includes an equal 1.25 h share of the 5.0 h defect-fix buffer. Combined: 31.50 person-hours, matching Section 6.3.

7.2 Rationale

- **Dependencies first.** TC-UPL-01 (valid upload → dataset_id) and the health checks of TC-MOD-06 are prerequisites for the model, job, metrics and results cases, so they run first on Day 1. Security probes (Akshat) and the UI and safety checks that need no completed job (Raghav) run in parallel on Day 1, so no one waits idle.
- **No hand-off delays inside an area.** Aman owns the full upload → API → performance chain, so the datasets and the timing scripts he prepares on Day 1 are reused in his own Day 2 performance cases.
- **Parallel tracks.** API-level testing (Aman, Aditya) and browser-level testing (Raghav) proceed simultaneously against the same deployment; security testing (Akshat) is independent of both.
- **Load balance.** Each member carries between 6.92 h and 9.00 h; the test lead carries the most because of setup and reporting.
- **Skill fit.** The asynchronous job lifecycle with failure injection (TC-JOB-05, TC-JOB-06) goes to one owner (Aditya) together with regression ownership, keeping the failure semantics with one person.

7.3 Schedule

Day 1 starts when a deployable build is available (the entry criteria in Section 4 are met).

Day	Member	Activities
Day 1	Akshat	Environment setup and smoke test; TC-SEC-01 – TC-SEC-05

Day	Member	Activities
Day 1	Aman	TC-UPL-01 (first, unblocks others), TC-UPL-02 – TC-UPL-08, TC-API-01 – TC-API-05; prepare perf_10k, perf_100k and large_noclass files
Day 1	Aditya	TC-MOD-06, TC-MOD-01 – TC-MOD-05
Day 1	Raghav	TC-RES-07, TC-SAFE-02, TC-SAFE-03; prepare tiny_4row.csv and reference_metrics.json
Day 2	Akshat	TC-SEC-06 – TC-SEC-10; defect triage
Day 2	Aman	TC-PERF-01 – TC-PERF-06
Day 2	Aditya	TC-JOB-01 – TC-JOB-06
Day 2	Raghav	TC-MET-01 – TC-MET-05, TC-RES-01 – TC-RES-06, TC-SAFE-01
Day 3	All	Defect fixes (buffer); re-test of fixed defects
Day 3	Aditya	Regression run (all High-priority and previously failed cases); portability smoke test on the second OS
Day 3	Raghav	TC-SAFE-04 usability walkthrough on the stabilised build
Day 3	Akshat	Test summary report and release recommendation

8. Item Pass/Fail Criteria

- A test case passes when every expected result is observed exactly as specified (status codes, body shapes, timing bounds, UI behavior).
- A test case fails on any deviation. A case that cannot be executed is recorded as blocked, separately, and is never counted as passed.
- A test item is accepted when all High-priority cases pass, at least 90% of all 57 cases pass, and no High-severity defect remains open.
- Every failure is logged as a defect with the test-case ID and the traced SRS requirement ID(s).

9. Suspension Criteria and Resumption Requirements

Suspend when: the Flask server or any microservice fails to start; more than 30% of executed cases are blocked by a single defect; or test data or the API-key fixture is unavailable.

Resume when the cause is fixed and the smoke test passes: GET /health → 200 on all services, TC-UPL-01 passes, and TC-JOB-02 reaches completed.

10. Test Deliverables

1. This test plan (STP-CCFD-1.1).
2. Test execution log — per-case result (pass/fail/blocked), timestamps, build hash, and actual versus estimated effort.
3. Defect reports — test-case ID, SRS requirement ID(s), severity, reproduction steps, owner.
4. Requirements traceability matrix (Appendix A), kept current with any test-case changes.
5. Test summary report — totals, pass rate, effort variance, open defects, release recommendation.

11. Environmental Needs

- Reference machine per SRS Section 2.4 (x86-64, 8 GB RAM, 2 GB free disk; Windows 10/11 or Ubuntu 20.04+), with access to a second machine or virtual machine running the other operating system for the portability smoke test.
- Python 3.10+, Flask 3.x and the libraries pinned in the developer README; quantum-circuit simulator libraries for TI-05.
- A current version of Chrome, Firefox or Edge with JavaScript enabled.
- The test datasets from Section 4, stored outside the system's dataset store.
- A dedicated test API key (never a production key).
- Single-host deployment; no external services required.

12. Responsibilities

Role	Member	Responsibility
Test lead	Akshat Agarwal (PES1UG24AM024)	Owns this plan and the schedule; environment setup; security cases (TC-SEC); defect triage; summary report and release recommendation
Test executor	Aman Priyansh (PES1UG24AM030)	Upload/validation, API contract and performance cases (TC-UPL, TC-API, TC-PERF); test data preparation
Test executor	Aditya Kalloli (PES1UG24AM015)	Model routing and job lifecycle cases (TC-MOD, TC-JOB); regression owner; portability smoke test
Test executor	Raghav Balaji (PES1UG23AM224)	Results/UI, metrics, safety and usability cases (TC-RES, TC-MET, TC-SAFE)
Faculty evaluator	Prof. C V Reddy	Reviews this plan and the test summary report

13. Risks and Contingencies

Risk	Impact	Contingency
600 s timeout (TC-JOB-06), 500 MB limit (TC-SEC-06) or 24 h retention (TC-SEC-09) impractical in lab time	Cases cannot run as specified	Configure lower values and record the substitution; the assertion logic is identical
QML simulator slow on the reference machine	TC-MOD-04 and TC-PERF-05 overrun	Use valid_1000.csv; only polling is timed, not model run time
Timing cases (TC-PERF-01 – TC-PERF-06) flake under background load	False failures	Median of 3 runs on an idle machine
Microservice contract drift between rounds	Integration failures	Re-run TC-MOD-06 before each system-test round
Build not yet stable on Day 1	Schedule slips	Add one fix-and-retest cycle (about 6 h); re-plan Day 3

Appendix A: Requirements Traceability Matrix

The matrix lists every SRS v1.2 requirement and the test cases that verify it. It is the reverse view of the Traceability field in each test case and is generated from those fields.

Requirement	Priority	Test cases	Method
REQ-UI-01	High	TC-RES-07	Test
REQ-UI-02	High	TC-RES-05	Test
REQ-UI-03	High	TC-JOB-02, TC-RES-04	Test
REQ-UI-04	Medium	TC-RES-07	Test
REQ-UI-05	Medium	TC-RES-01, TC-RES-02	Test
REQ-UI-06	Low	TC-RES-07	Test
REQ-FD-01	High	TC-RES-07	Test
REQ-FD-02	High	TC-SEC-06, TC-UPL-01, TC-UPL-06	Test
REQ-FD-03	High	TC-UPL-01, TC-UPL-02, TC-UPL-03, TC-UPL-04, TC-UPL-07, TC-UPL-08	Test
REQ-FD-04	High	TC-SEC-05, TC-SEC-06, TC-UPL-02, TC-UPL-03, TC-UPL-04, TC-UPL-05, TC-UPL-06, TC-UPL-07	Test
REQ-FD-05	Medium	TC-UPL-01	Test
REQ-FD-06	High	TC-MOD-01, TC-MOD-02, TC-MOD-03, TC-MOD-04, TC-MOD-05	Test
REQ-FD-07	High	TC-MOD-02	Test
REQ-FD-08	High	TC-MOD-03	Test
REQ-FD-09	High	TC-MOD-04	Test
REQ-FD-10	Medium	TC-MOD-02, TC-MOD-03, TC-MOD-04, TC-MOD-06	Test
REQ-FD-11	High	TC-MOD-02, TC-MOD-03, TC-MOD-04, TC-JOB-04	Test
REQ-FD-12	High	TC-JOB-01, TC-JOB-02, TC-JOB-04	Test
REQ-FD-13	High	TC-RES-03	Test
REQ-FD-14	High	TC-JOB-05, TC-JOB-06	Test
REQ-FD-15	High	TC-MET-01, TC-MET-04, TC-MET-05	Test
REQ-FD-16	High	TC-MET-02, TC-MET-04, TC-MET-05	Test
REQ-FD-17	Medium	TC-MOD-02, TC-MOD-03, TC-MOD-04, TC-MET-03	Test
REQ-FD-18	High	TC-RES-01, TC-RES-02	Test
REQ-FD-19	High	TC-MET-03	Test
REQ-FD-20	High	TC-JOB-05, TC-RES-04	Test
REQ-FD-21	Medium	TC-RES-06	Test
REQ-FD-22	Medium	TC-JOB-04	Test

Requirement	Priority	Test cases	Method
REQ-API-01	High	TC-UPL-01, TC-API-01	Test
REQ-API-02	High	TC-SEC-03, TC-MOD-05, TC-JOB-01, TC-API-02	Test
REQ-API-03	High	TC-SEC-10, TC-JOB-03, TC-API-03	Test
REQ-API-04	High	TC-SEC-01, TC-SEC-04, TC-MOD-05, TC-JOB-03, TC-API-04, TC-API-05	Test
REQ-API-05	High	TC-API-01, TC-API-02, TC-API-03, TC-API-05	Inspection of routes.py plus the TC-API contract cases
NFR-01	High	TC-UPL-01, TC-PERF-01	Test
NFR-02	High	TC-PERF-02	Test
NFR-03	Medium	TC-PERF-03	Test
NFR-04	Medium	TC-PERF-04	Test
NFR-05	Medium	TC-PERF-05	Test
NFR-06	Low	TC-JOB-05	Test
NFR-07	Medium	TC-PERF-06	Test
NFR-S01	High	TC-SAFE-02	Test
NFR-S02	High	TC-SAFE-01	Test
NFR-S03	Medium	TC-SAFE-03	Test
NFR-Q01	Medium	TC-SAFE-04	Test
NFR-Q02	Medium	—	Inspection of the microservice contract (SADS 3.2) plus TC-MOD-06
NFR-Q03	Medium	TC-PERF-05	Test
NFR-Q04	Low	—	Smoke test run on Windows and on Ubuntu (Section 4)
NFR-Q05	Medium	TC-MOD-06	Test
SR-01	High	TC-SEC-05, TC-SEC-06	Test
SR-02	High	TC-SEC-04, TC-MOD-05	Test
SR-03	High	TC-SEC-01, TC-SEC-02, TC-SEC-03, TC-SEC-10	Test
SR-04	High	TC-SEC-08	Test
SR-05	Medium	TC-SEC-09	Test
SR-06	Medium	TC-SEC-07	Test
SO-01	—	TC-SEC-01, TC-SEC-02, TC-SEC-03, TC-SEC-05, TC-SEC-06, TC-SEC-07, TC-SEC-08, TC-SEC-09, TC-SEC-10	Through SR-01, SR-03, SR-04, SR-05, SR-06

Requirement	Priority	Test cases	Method
SO-02	—	TC-SEC-01, TC-SEC-02, TC-SEC-03, TC-SEC-07, TC-SEC-10	Through SR-03, SR-06
SO-03	—	TC-SEC-04, TC-SEC-05, TC-SEC-06, TC-MOD-05	Through SR-01, SR-02
BR-01	—	TC-UPL-02	Test
BR-02	—	TC-MOD-01	Test
BR-03	—	TC-MOD-02, TC-MOD-03, TC-MOD-04, TC-MET-03	Test
BR-04	—	TC-JOB-05	Test
BR-05	—	TC-SAFE-01, TC-SAFE-02	Test

Coverage summary: of the 59 SRS requirements and business rules, 57 are verified by at least one test case; NFR-Q02 (maintainability) is verified by design inspection of the uniform microservice contract together with the TC-MOD-06 contract checks, and NFR-Q04 (portability) by the smoke test run on both operating systems (Section 4). All 33 High-priority requirements are covered by at least one test case, and the three security objectives are verified through the security requirements that support them.

Appendix B: Glossary

Term	Meaning
STP	Software Test Plan (this document, STP-CCFD-1.1).
Est.	Estimated hands-on minutes for one tester to set up, execute, verify and log the case.
Smoke test	Minimal pass (health checks, TC-UPL-01 and TC-JOB-02) proving that a build is testable.
Blocked	A case that could not be executed because of environment or build problems.
Upload completion	The moment the Web Server has received the complete upload body (SRS Section 5.1).
p95	95th percentile of measured response times.
Person-hour	One hour of work by one team member.
Traceability	Mapping between each test case and the SRS requirement(s) it verifies.

Project Team

SRN	Name
PES1UG24AM024	Akshat Agarwal
PES1UG24AM030	Aman Priyansh
PES1UG24AM015	Aditya Kalloli
PES1UG23AM224	Raghav Balaji